Retrieval Engine Guide from Zero to 100

A readable study guide for the competition notebook and report

Purpose

This guide explains the retrieval project notebook from the ground up. It starts with the basic language of information retrieval, then moves to TF-IDF, BM25+, dense embeddings, category classification, dominant-category filtering, evaluation, and performance engineering.

Be carefull

Two common misconceptions should be corrected from the start.

- **TF-IDF is not just repeated-word counting.** It weights within-document frequency by how rare the term is across the corpus.
- **BM25+ is not mainly a tokenizer or stopword remover.** BM25+ is a probabilistic lexical ranking function with term-frequency saturation and document-length normalization. Tokenization and stopword removal are optional preprocessing choices, not the core formula.

Guide map

```
START
  |
  +--> Section 1: how to read the notebook as a system
  |
  +--> Section 2: retrieval basics from zero
  |
  +--> Section 3: preprocessing and text construction
  |
  +--> Section 4: TF-IDF in an academic but readable way
  |
  +--> Section 5: BM25+ and why it usually beats plain TF-IDF lexically
  |
  +--> Section 6: dense embeddings and semantic retrieval
  |
  +--> Section 7: classifier + dominant-category filter
  |
  +--> Section 8: evaluation metrics and score interpretation
  |
```

```
+--> Section 9: caching, chunking, argpartition, and runtime design
|
`--> Appendix: formulas, oral-exam answers, and a tuning checklist
```

March 26, 2026

1 Start here: data science basics for retrieval

New to the topic?

Retrieval is a ranking problem on text. For each query, the system must score many candidate documents and place the most useful ones at the top. This page gives the background vocabulary and mental model you need before the notebook starts using terms such as TF-IDF, BM25+, embeddings, and Recall@K.

- **What the task is.** In ordinary classification, one input usually gets one label. In retrieval, one query is compared against many documents, and the output is an ordered list. So the goal is not just "is this document good or bad?" but "which documents should appear first?"
- **Data science loop.** The loop is: define the question, collect the data, clean and normalize the text, turn text into numeric representations, choose a scoring model, rank documents, evaluate on held-out queries, inspect failures, and improve the pipeline. The notebook follows exactly this pattern.
- **The main dataset objects.** A *document* belongs to the corpus we search through. A *query* is the user need we want to satisfy. A *sample* can mean one document row, one query row, or one query-document pair depending on the stage of the pipeline. A *feature* is any numeric signal we compute from text. A *label* tells us what the correct answer is; in retrieval, that usually means whether a document is relevant to a query.
- **Why train, validation, and test are separated.** We use *train* data to fit parameters, *validation* data to compare design choices such as `top_k` or preprocessing, and *test* data only for the final unbiased check. Looking at test results too early causes *data leakage*: the system starts to indirectly adapt to the test set instead of truly generalizing.
- **Text cleaning and normalization.** Raw text is messy: uppercase/lowercase differences, repeated spaces, punctuation variants, tags, ids, and separators like `-` or `/`. Normalization makes equivalent strings look more alike. In this notebook, the cleaning is intentionally light so that technical tokens such as version numbers, package names, and error codes are not destroyed.
- **Tokenization.** Tokenization breaks text into smaller units called *tokens*. With simple tokenization, the sentence "GPU-driver error 404" can become tokens such as `gpu`, `driver`, `error`, and `404`. Once text is tokenized, the system can count tokens, weight them, or map them into embeddings.
- **Vectorization.** Computers do not compare raw paragraphs directly; they compare numbers. Vectorization maps a query or document to a numeric representation.
  - *Sparse lexical methods* represent exact token evidence. Bag-of-words, TF-IDF, and BM25+ all reward overlap in words or terms. They work especially well when exact wording matters, such as error messages, library names, commands, or product models.
  - *Dense semantic methods* represent meaning in a compact vector called an embedding. They can rank two texts near each other even when they use different surface words, which helps with paraphrases and vocabulary mismatch.
- **Scoring and ranking.** After vectorization, the system computes a relevance score for every query-document pair. For TF-IDF or embeddings, that score is often cosine similarity or dot product. For BM25+, it comes from a lexical scoring formula that uses term frequency, document frequency, and document length. The exact numeric value matters less than the ordering it produces: higher score means the document should appear earlier in the ranking.
- **Top-k retrieval.** A retriever usually does not keep every scored document. It keeps only the best K documents, often written `top_k`. Small K is faster but may miss relevant documents; large K improves coverage but increases compute and can lower precision.

- **Evaluation.** We judge the system on held-out queries whose relevant documents are known. *Recall@K* asks "did we retrieve the relevant documents somewhere in the top K?" *Precision@K* asks "how many of the top K were actually relevant?" *MAP* and *MRR* care not only about whether relevant documents appear, but also how early they appear in the ranked list.
- **Mental model for the rest of the guide.** Whenever a later section introduces TF-IDF, BM25+, embeddings, or category filtering, ask three questions: what representation is being built, how are scores computed, and which evaluation metric improved? That simple checklist is enough to follow most of the notebook.

A tiny end-to-end example

Suppose the query is "python socket timeout". A lexical method such as TF-IDF or BM25+ will strongly favor a document containing the exact words `python`, `socket`, and `timeout`. A dense embedding model may also rank a document about "network connection delays in Python" even if it does not repeat all three words exactly.

So the full pipeline is:

1. normalize and tokenize the query and documents,
2. turn them into sparse or dense numeric representations,
3. compute a score between the query and every candidate document,
4. sort documents by score and check whether the truly relevant ones rise near the top.

This is the same pattern repeated throughout the notebook; only the representation and scoring rule change.

Tip

If a later section feels abstract, come back to this page and identify the current step of the pipeline: cleaning, tokenization, vectorization, scoring, ranking, or evaluation. Once you know the step, the jargon becomes much easier to decode.

With these basics, the later sections on TF-IDF, BM25+, embeddings, category filtering, and evaluation should read like variations of one common pipeline instead of isolated ideas.

2 The retrieval problem

2.1 What is information retrieval?

Information retrieval asks a simple question: given a query, which documents in a large collection should be ranked highest? In this project, the collection contains technical forum documents, and each query is a short technical problem statement. The system returns a ranked list of document identifiers.

The central objects are:

$$\mathcal{D} = \{d_1, d_2, \ldots, d_N\} \quad \text{(documents)}, \qquad \mathcal{Q} = \{q_1, q_2, \ldots, q_M\} \quad \text{(queries)}.$$

For one query q, the retriever produces an ordering

$$\pi_q = (d_{i_1}, d_{i_2}, \ldots, d_{i_K}),$$

where earlier positions are supposed to be more relevant.

2.2 What does “relevant” mean?

A document is relevant to a query if it helps answer that query according to the competition ground truth. Relevance is not identical to textual overlap. A relevant document can use different words from the query but still discuss the same underlying issue. This distinction is exactly why sparse lexical methods and dense semantic methods behave differently.

2.3 What is a ranking score?

A ranking function assigns a real number to each query-document pair:

$$\text{score}(q, d) \in \mathbb{R}.$$

The system sorts documents by descending score. Different retrieval methods define this score differently:

- TF-IDF usually uses cosine similarity between sparse vectors.
- BM25+ uses a token-level lexical relevance formula.
- Dense retrieval uses similarity between learned dense vectors.

2.4 What is top-k?

`top_k` is the number of documents retained for each query after the first-stage retriever. Small K can miss relevant documents. Large K improves recall but usually hurts precision and increases compute and output size.

One-query view of the pipeline

```
query q
  |
  v
[text normalization]
  |
  v
[first-stage retriever]
  |------------------------------|
  | TF-IDF     BM25+   Embedding |
  |------------------------------|
  |
  v
ranked list of top_k documents
  |
  v
optional category-based filtering
  |
  v
final ranked submission / offline evaluation
```

2.5 Why lexical and semantic retrieval differ

Lexical retrieval rewards exact token overlap. It shines when the wording matters: error strings, API names, product models, flags, codes, and identifiers. **Semantic retrieval** uses learned embeddings to place

related meanings near one another, so it succeeds when the query paraphrases the document instead of reusing the same words.

Warning

A query and a relevant document can talk about the same issue while sharing few exact words. That failure mode is often called **vocabulary mismatch** or **semantic mismatch**. This is the main reason embedding retrieval became the final active method in the notebook.

3 Dataset, text construction, and preprocessing

3.1 Dataset objects in the notebook

The project materials describe a collection of 216,041 documents, 327 training queries, 141 test queries, and a training relevance file. Documents belong to five categories: `tex`, `unix`, `gaming`, `programmers`, and `android`. That category signal later becomes useful for classification and filtering.

3.2 Why strict schema checks matter

The notebook performs early validation of columns and identifier uniqueness. This is not cosmetic. Retrieval systems break quietly when ids are duplicated, when expected fields are missing, or when train and test schemas differ slightly.

Conceptually, early validation prevents these classes of bugs:

- a document missing its text field,
- duplicate ids that later corrupt ranking output,
- category labels absent during classifier training,
- broken merges between results and relevance judgments.

3.3 How text is constructed

The notebook builds retrieval documents from

`title + text + tags`,

while retrieval queries use

`title + text`.

For the classifier, the notebook can use query tags as well. This distinction matters because the retriever and classifier are not solving exactly the same subproblem.

Why combine multiple fields?

A forum post title is often concise and highly informative, the body provides detailed context, and tags provide compact topical labels. Merging them can increase retrieval signal. But you should merge them consistently across train and test to keep the feature space stable.

3.4 Normalization policy

The notebook uses light, shared normalization instead of aggressive linguistic processing. In simplified form, the policy is:

1. replace separators `-`, `\_`, and `/` with spaces,
2. lowercase the text,
3. collapse repeated whitespace,
4. strip leading and trailing space.

This produces consistent text while preserving most lexical information.

Listing 1: Conceptual normalization logic

```
def normalize_text(text):
    text = replace_separators(text, chars='-_/')
    text = text.lower()
    text = collapse_whitespace(text)
    return text.strip()
```

3.5 Tokenization in this notebook

The lexical token pattern is

`[a-z0-9]+`.

That means tokens are sequences of lowercase letters and digits. So numbers are not automatically removed; they are kept whenever they match the pattern.

Warning

This notebook does **not** implement classical stopword removal as a core preprocessing step. Therefore words such as `the` or `is` are not removed just because BM25+ is used. They may receive low discriminative weight, but they are still tokens unless you explicitly filter them.

3.6 Why use shared preprocessing across all retrievers?

If TF-IDF, BM25+, and embeddings receive differently normalized text, comparison becomes noisy. You no longer know whether a performance change came from the model or from preprocessing differences. Shared normalization is therefore an experimental control.

4 TF-IDF from zero

4.1 The bag-of-words idea

TF-IDF starts from a sparse representation. Imagine a vocabulary

$$V = \{t_1, t_2, \ldots, t_{|V|}\}.$$

Each document becomes a vector of size $|V|$. Most entries are zero because each document only uses a small subset of all possible words. This is why TF-IDF is called a **sparse** representation.

4.2 Term frequency

For a term t and document d, term frequency $\mathrm{tf}(t, d)$ measures how often t appears in d. A larger count usually suggests the term is more central to that document.

But raw repetition alone is not enough. If a word occurs in almost every document, it is not very helpful for distinguishing relevant from irrelevant documents.

4.3 Inverse document frequency

Let N be the total number of documents and $\mathrm{df}(t)$ the number of documents containing term t. Inverse document frequency reduces the importance of overly common terms and boosts rarer terms. A standard form is:

$$\mathrm{idf}(t) = \log\left(\frac{N}{\mathrm{df}(t)}\right).$$

Different libraries use smoothed variants, but the intuition is always the same: rare terms carry more discriminative information.

4.4 Putting TF and IDF together

A simple TF-IDF weight is

$$\mathrm{tfidf}(t, d) = \mathrm{tf}(t, d) \cdot \mathrm{idf}(t).$$

So TF-IDF is not merely counting repeated words. It answers a more refined question:

> Is this term frequent in this document *and* relatively rare in the corpus?

4.5 Query and document vectors

The query is also turned into a TF-IDF vector. Then the model computes similarity between the sparse query vector and each sparse document vector.

The most common similarity is cosine similarity:

$$\cos(\theta) = \frac{\mathbf{q} \cdot \mathbf{d}}{\|\mathbf{q}\| \, \|\mathbf{d}\|}.$$

This measures angle rather than raw length, which makes the comparison more stable when queries and documents have different sizes.

TF-IDF intuition

```
rare term in query + rare term in document  --> strong signal
common term in query + common term in document --> weak signal
no exact overlap --> almost no lexical signal
```

4.6 Why the notebook uses unigrams and bigrams

The notebook uses `ngram_range=(1,2)`. That means it includes:

- unigrams: single tokens such as `kernel`
- bigrams: adjacent token pairs such as `kernel panic`

Bigrams help preserve short phrases and can improve lexical specificity.

The notebook also uses `min_df=2`, meaning terms that appear in only one document are pruned. This reduces very rare noise, though the vectorizer can fall back to `min_df=1` if pruning removes everything in an edge case.

4.7 Strengths of TF-IDF

- simple and fast,
- easy to inspect,
- strong on exact technical terms,
- good as a baseline and debugging instrument.

4.8 Weaknesses of TF-IDF

- weak on paraphrases,
- weak when query and document use synonyms,
- less robust when documents are very long and heterogeneous,
- still largely dependent on direct overlap.

Tip

In oral or written explanation, a clean sentence is: "TF-IDF is a sparse lexical weighting scheme that emphasizes terms that are frequent within a document but rare across the collection, and it is typically paired with cosine similarity for ranking."

5 BM25+ from zero

5.1 Why BM25 exists if we already have TF-IDF

TF-IDF is a good baseline, but it does not model two practical effects very elegantly:

1. repeated occurrences of a term should help, but with diminishing returns,
2. long documents should not win only because they contain more words.

BM25 was designed to address these issues more directly.

5.2 The core BM25 idea

For each query term t, BM25 computes a contribution to the score of document d. A standard BM25+ form is

$$\text{score}(q,d) = \sum_{t \in q} \text{idf}(t) \left(\frac{f(t,d)(k_1+1)}{f(t,d) + k_1 \left(1 - b + b \frac{|d|}{\text{avgdl}}\right)} + \delta \right),$$

where:

- $f(t,d)$ is the frequency of term t in document d,

- $|d|$ is document length,
- avgdl is average document length in the corpus,
- k_1 controls term-frequency saturation,
- b controls document-length normalization,
- δ is the BM25+ offset.

5.3 Term-frequency saturation

BM25 does not reward repeated terms linearly forever. The first few appearances matter more than the fiftieth. This is called **term-frequency saturation**. It reflects the intuition that after a point, additional repetitions add less new evidence.

5.4 Length normalization

If two documents mention the same important term equally well, a much longer document should not automatically be preferred just because it contains more total words. The b parameter adjusts for this by comparing document length to average document length.

5.5 What BM25+ adds

BM25+ adds the δ term so that long documents are not overly penalized when they do contain a query term. It is a refinement of BM25, especially helpful in some long-document settings.

5.6 How to interpret the notebook parameters

The notebook uses:

$$k_1 = 1.5, \qquad b = 0.75, \qquad \delta = 1.0.$$

These are reasonable, standard-looking defaults:

- $k_1 = 1.5$: moderate saturation,
- $b = 0.75$: substantial but not extreme length normalization,
- $\delta = 1.0$: positive BM25+ adjustment.

5.7 Correcting the stopword misconception

Your intuition that BM25 cares about common words is directionally related to reality, but the mechanism is different from "BM25 removes the, a, is".

The correct explanation is:

- BM25 is still a lexical matching model based on tokens.
- Common terms often receive lower idf, so they contribute less to ranking.
- Tokenization and stopword removal happen *before* scoring if the pipeline designer chooses them.
- In this notebook, the tokenizer keeps lowercase alphanumeric tokens, including numbers, and there is no dedicated stopword-removal stage in the shared preprocessing.

5.8 Why BM25+ can beat TF-IDF lexically

On long and uneven documents, BM25+ often gives more realistic lexical scores because it combines term rarity, saturation, and length normalization in a more retrieval-specific way than plain TF-IDF cosine similarity.

BM25+ intuition in one line

```
TF-IDF asks: which rare terms overlap?
BM25+ asks: which rare terms overlap, with sensible diminishing returns and document-
    length correction?
```

5.9 What BM25+ still cannot solve

BM25+ is still lexical. If a relevant document says `application freezes` and the query says `program hangs`, BM25+ may still miss the connection if the shared token overlap is weak.

6 Dense embeddings and semantic retrieval

6.1 Why move from sparse to dense representations?

Sparse vectors have one dimension per vocabulary item. Dense vectors instead have a fixed low dimension and are learned so that semantically similar texts land close to one another in vector space.

In the project notebook, the active dense model is `all-MiniLM-L6-v2`. The report describes document and query vectors of dimension 384.

6.2 What an embedding is

An embedding is a dense vector representation:

$$\mathbf{e}(x) \in \mathbb{R}^{384}.$$

The important idea is not the exact dimension; it is that semantic information is distributed across the vector coordinates.

6.3 How dense retrieval works

The notebook encodes every document into a dense vector and stores the resulting matrix. Each query is also encoded into a dense vector. Then the retriever computes similarity between the query vector and all document vectors.

If vectors are L2-normalized, cosine similarity and dot product become equivalent up to scale:

$$\cos(\theta) = \mathbf{e}(q)^\top \mathbf{e}(d) \quad \text{when both vectors have unit norm.}$$

6.4 Why dense retrieval helps here

Dense retrieval is especially strong when:

- the query paraphrases the relevant document,

- the same issue is described with different wording,
- abbreviations and alternate phrasing appear,
- exact lexical overlap is sparse.

Semantic shift

The notebook moves from lexical matching to semantic matching. Instead of asking only "which tokens overlap?", the dense model asks "which texts are close in learned meaning space?".

6.5 Why embeddings are expensive

Dense retrieval usually improves recall, but it introduces computational cost:

- encoding all documents is expensive,
- storing the embedding matrix takes memory,
- scoring query vectors against all document vectors can be slow.

This is why the notebook invests heavily in caching and chunked scoring.

6.6 Batching and chunking

The notebook uses an embedding batch size for encoding and a query chunk size for scoring. These are engineering controls, not retrieval theory. They do not change the mathematical objective; they change runtime and memory behavior.

If you score all queries against all documents at once, you may create a huge dense score matrix. Chunking processes manageable query groups instead.

6.7 What dense retrieval does not magically solve

Dense retrieval is not perfect.

- It can be slower.
- It can require more memory.
- It can produce semantically plausible but category-wrong documents.
- It may need post-filtering or reranking for best precision.

Warning

Dense retrieval is usually a recall tool first. In many systems, it brings more relevant candidates into the top-K, but later stages are still needed to clean noise.

7 The category classifier and dominant-category filter

7.1 Why add a classifier to a retriever?

The retriever finds documents that look relevant. The classifier predicts which topical category a query belongs to. These are related but not identical tasks.

In this notebook, the category classifier is trained with TF-IDF features and a linear support vector machine:

- features: TF-IDF vectors,
- classifier: `LinearSVC`,
- label: one category per example.

7.2 What a linear SVM is doing here

A linear SVM learns a separating hyperplane between classes in feature space. With a one-vs-rest or multi-class scheme, it can predict one category for each query from its TF-IDF representation.

A clean academic description is:

> The classifier is a linear discriminative model over sparse TF-IDF features, trained to predict the most likely forum category of each query.

7.3 Why this can improve the pipeline

Suppose dense retrieval returns mostly good documents but also mixes in some documents from semantically nearby yet wrong categories. If the query is really `android` and many deep-ranked documents drift into `programmers`, a category filter can remove part of that noise.

7.4 The notebook's dominant-category filter

The notebook does something specific and clever:

1. take the top N retrieved documents for one query, with $N = 20$,
2. count the categories in that top window,
3. find the dominant category,
4. keep only documents from that category in the final ranked list.

So the top retrieved documents vote for a category, and the rest of the ranked list is filtered accordingly.

Dominant-category filter

```
Top 20 retrieved docs for one query
  |
  +--> android: 11
  +--> programmers: 5
  +--> unix: 4
  |
  `--> dominant category = android

Keep only documents labeled android in the final list.
```

7.5 Why this helps

It can clean the long tail of a large `top_k` ranking. If the first-stage retriever returns 7,500 documents, later positions inevitably contain noise. A dominant-category filter removes some of that noise while preserving the general ranking order inside the chosen category.

7.6 What the risk is

If the dominant category is wrong for a query, the filter can remove relevant documents from the correct category. So the filter trades off robustness against category drift with the risk of category over-commitment.

Tip

You can describe the post-filter as a **high-precision biasing step**: it assumes that the category distribution of the top window is informative enough to prune later noise.

8 Evaluation metrics and how to read them

8.1 Recall@K

Recall asks: among all truly relevant documents, how many did we recover?

$$\text{Recall@K} = \frac{|R_q \cap \hat{R}_{q,K}|}{|R_q|},$$

where R_q is the set of truly relevant documents and $\hat{R}_{q,K}$ is the predicted top-K set.

High recall means the system is good at not missing relevant material.

8.2 Precision@K

Precision asks: among the documents we returned, how many are truly relevant?

$$\text{Precision@K} = \frac{|R_q \cap \hat{R}_{q,K}|}{|\hat{R}_{q,K}|}.$$

If K is very large, precision often becomes small, because the denominator grows quickly.

8.3 MRR@K

Mean Reciprocal Rank focuses on the first relevant hit. For one query, if the first relevant document appears at rank r_q, then the reciprocal rank is

$$\frac{1}{r_q}.$$

Averaging across queries gives MRR. High MRR means useful documents appear early.

8.4 Accuracy

In this project, accuracy refers to the category classifier. It asks how often the predicted category matches the true category.

8.5 The combined score

The report averages four quantities:

$$\text{Score} = \frac{1}{4}(\text{Recall} + \text{Precision} + \text{MRR} + \text{Accuracy}).$$

This means the system is not evaluated only as a retriever; it is evaluated as a retrieval-plus-category pipeline.

8.6 How to interpret a very low precision with high recall

Because the active configuration uses a huge initial $K = 7500$, precision is expected to be numerically small. That does not automatically mean the system is poor. It may simply reflect the fact that you intentionally retained a large candidate set to protect recall.

The more meaningful interpretation is joint:

- high recall says relevant documents are usually present,
- reasonable MRR says useful documents appear early,
- classifier accuracy and filtering help control category noise,
- very low precision is partly a mathematical consequence of a large output list.

9 Performance engineering: why the notebook does not stop at theory

9.1 Why engineering matters here

A theoretically good retriever is not enough if every iteration takes too long. The report notes that the embedding pipeline initially approached a severe runtime bottleneck, so the notebook adds several engineering optimizations.

9.2 Disk caching

Dense document embeddings are expensive to recompute. The notebook fingerprints relevant dataframes and configuration, then saves reusable artifacts such as embedding matrices and classical indexes to disk.

This is important because the expensive step is often document encoding, not query encoding. Once document embeddings are cached, repeated runs become dramatically cheaper.

9.3 In-memory caching

The notebook also keeps objects in memory dictionaries during a live session. This avoids unnecessary repeated loads from disk after the first use.

9.4 Chunked query scoring

Instead of scoring all queries against all documents in one giant matrix multiplication, the notebook scores query chunks, for example 32 at a time. This controls peak memory usage.

9.5 Partial sorting with `np.argpartition`

A full sort of all document scores for every query is expensive when you only need the best K items. `np.argpartition` is a partial-selection method. It finds the top part more cheaply than a full exact sort of the entire array. You can then sort only that smaller candidate slice.

Why partial sorting is faster

```
Need top 7,500 docs out of 216,041?

Full sort:
  sort all 216,041 scores

Partial selection:
  find only the best 7,500 region first
  then sort that region
```

9.6 Reusing one max-K ranking

During offline sweeps, if you compute a correct sorted top-7,500 ranking, then top-100, top-500, or top-1,000 prefixes can be obtained by truncation. The notebook reuses that fact instead of recomputing rankings for every smaller K.

9.7 The engineering lesson

The notebook is not only a modeling notebook. It is an engineering report. Its final quality comes from both retrieval choices and systems design.

10 Notebook-aligned walkthrough

10.1 How the notebook is organized

The saved notebook is organized in a clean progression:

1. project overview,
2. explanation of why embeddings are the default,
3. `top_k` discussion,
4. method comparison,
5. imports and configuration,
6. data loading,
7. preprocessing,
8. index construction and caching,
9. retrieval functions,
10. evaluation logic,
11. experiment loop,

12. test submission generation,
13. conclusion.

10.2 What each code region is conceptually doing

Notebook region	Conceptual role
Imports and config	Defines global settings: active model, `top_k`, tokenizer pattern, TF-IDF parameters, BM25+ parameters, embedding model name, cache directories.
Data loading	Reads raw JSON files, validates required columns, checks ids, and enforces reproducibility assumptions.
Preprocessing	Normalizes text and builds unified `content` fields for documents and queries.
Index construction	Fits TF-IDF, builds BM25+, loads the sentence-transformer, and prepares or loads cached embeddings.
Retrieval functions	Converts one method choice into a ranked list of documents for each query.
Evaluation logic	Computes Recall, Precision, MRR, Accuracy, and their average score.
Experiment loop	Runs the selected models, applies the dominant-category filter, and collects summary tables.
Submission generation	Runs the final model on the test queries and writes the output CSV.

10.3 What is "active" and what is "implemented"

A crucial distinction in the project report is this:

- TF-IDF retrieval is implemented, but not the final active submission method.
- BM25+ retrieval is implemented, but not the final active submission method.
- Embedding retrieval is both implemented and active.
- The category classifier and dominant-category filter are active support components.

That is an academically strong experimental structure: keep simpler baselines for comparison, but submit the method that actually wins on the observed task.

11 A complete mental model of the full pipeline

End-to-end mental model

```
1. Load documents, train queries, test queries, qrels, and submission template.
2. Validate schemas and ids.
3. Normalize text consistently.
4. Build document content and query content.
5. Train the query category classifier (TF-IDF + LinearSVC).
6. Build retrieval artifacts for TF-IDF, BM25+, and/or embeddings.
7. Run retrieval for each query.
8. Keep a large top_k candidate list to protect recall.
9. Use top-20 category composition to choose a dominant category.
10. Filter the candidate list by that dominant category.
11. Evaluate on train queries.
```

```
12. Run the final model on test queries and export CSV.
```

One-sentence summary

The final system is a dense semantic first-stage retriever, supported by a sparse linear category classifier and a dominant-category filter, all wrapped in a cache-aware evaluation and submission pipeline.

12 Common misunderstandings corrected

(Misunderstanding 1)

"TF-IDF finds repeated words."
Only partly. It weights within-document frequency by inverse document frequency across the corpus. Repetition matters, but rarity matters too.

(Misunderstanding 2)

"BM25 removes stopwords."
Not inherently. BM25 is a ranking formula. Stopword removal is a preprocessing design choice. In this notebook, there is no dedicated stopword-removal step in the shared normalization pipeline.

(Misunderstanding 3)

"Embeddings are just another kind of token count."
No. Dense embeddings are learned vector representations intended to capture semantic proximity, including paraphrase-level similarity.

(Misunderstanding 4)

"Low precision means the system failed."
Not necessarily. With very large K, precision can be numerically tiny while recall and MRR remain strong. Metric interpretation depends on task design.

13 How to explain this project academically in class or in a report

13.1 Short version

"We implemented three first-stage retrievers: TF-IDF, BM25+, and dense embeddings. TF-IDF and BM25+ serve as sparse lexical baselines, while the final active system uses a sentence-transformer embedding retriever because it better addresses vocabulary mismatch. We then combine retrieval with a TF-IDF + LinearSVC category classifier and a dominant-category post-filter to clean the long tail of a large top-K ranking. The pipeline is optimized with artifact caching, chunked query scoring, and partial top-K selection."

13.2 Longer version

"The project begins with strict schema validation and shared normalization to make model comparisons trustworthy. Documents are represented as title + body + tags, while queries use title + body for retrieval and can include tags for category prediction. TF-IDF provides a sparse cosine baseline, BM25+ provides a stronger lexical baseline with term-frequency saturation and document-length normalization, and dense embeddings provide a semantic retriever robust to paraphrase. Because dense retrieval is computationally expensive, the notebook caches document embeddings and scores queries in chunks. After retrieval, a lightweight linear category classifier predicts topical structure, and a dominant-category filter uses the top retrieved window to suppress cross-category noise before evaluation and submission generation."

14 A practical tuning checklist

1. Verify data loading and ids before changing models.
2. Keep normalization shared across methods.
3. Establish TF-IDF as the sanity-check baseline.
4. Compare BM25+ against TF-IDF to isolate lexical improvements.
5. Move to embeddings when semantic mismatch is the main failure mode.
6. Cache document embeddings immediately.
7. Tune `top_k` for recall first, precision second.
8. Inspect whether category filtering removes useful documents or only noise.
9. Report Recall, Precision, MRR, and Accuracy together rather than in isolation.
10. Reuse max-K results for smaller K sweeps when possible.

A Formula sheet

TF-IDF

$$\text{tfidf}(t, d) = \text{tf}(t, d) \cdot \text{idf}(t)$$

$$\text{idf}(t) = \log\left(\frac{N}{\text{df}(t)}\right)$$

Cosine similarity

$$\cos(\theta) = \frac{\mathbf{q} \cdot \mathbf{d}}{\|\mathbf{q}\| \, \|\mathbf{d}\|}$$

BM25+

$$\text{score}(q, d) = \sum_{t \in q} \text{idf}(t) \left(\frac{f(t, d)(k_1 + 1)}{f(t, d) + k_1 \left(1 - b + b \frac{|d|}{\text{avgdl}}\right)} + \delta \right)$$

Recall@K

$$\text{Recall@K} = \frac{|R_q \cap \hat{R}_{q,K}|}{|R_q|}$$

Precision@K

$$\text{Precision@K} = \frac{|R_q \cap \hat{R}_{q,K}|}{|\hat{R}_{q,K}|}$$

MRR

$$\text{MRR} = \frac{1}{|Q|} \sum_{q \in Q} \frac{1}{r_q}$$

Combined score

$$\text{Score} = \frac{1}{4}(\text{Recall} + \text{Precision} + \text{MRR} + \text{Accuracy})$$

B Ten oral-exam questions with model answers

1. **Why is TF-IDF sparse?** Because each vector has one dimension per vocabulary item, and most terms are absent from a given document.
2. **Why is BM25+ usually better than raw TF-IDF for lexical retrieval?** Because it includes term-frequency saturation and document-length normalization, which better reflect retrieval behavior on uneven documents.
3. **Why are embeddings useful here?** Because relevant technical documents may use different wording from the query, and dense embeddings are more robust to paraphrase.
4. **Why not submit TF-IDF if it is simpler?** Because a simple baseline is valuable for comparison, but the final choice should follow empirical performance on the task.
5. **Why use a category classifier?** To estimate the topical region of the query and reduce cross-category noise in deep rankings.
6. **What does the dominant-category filter assume?** It assumes the category composition of the top retrieval window is informative enough to guide pruning of later results.
7. **Why is precision small when K is large?** Because the denominator includes many returned documents; even strong recall can coexist with low precision at large K.
8. **Why cache embeddings?** Because document encoding is expensive and does not need to be repeated every run.
9. **Why use query chunking?** To control memory usage during dense scoring.
10. **Why use partial sorting?** Because you only need the top part of the ranking, not a full exact sort of all scores.

C Final takeaway

What you should remember most

This project is not just "TF-IDF versus BM25 versus embeddings." It is a full retrieval system. The final notebook combines semantic first-stage retrieval, sparse linear category prediction, category-aware post-filtering, and strong runtime engineering. The best way to describe it academically is: *a cache-aware hybrid retrieval pipeline with dense semantic ranking and lightweight categorical control.*